Testing with RSpec

RSpec is an alternative to MiniTest, which Rails uses. It's different in almost every way, and many developers prefer it. Here's what one of our existing tests might look like written in RSpec:

```
RSpec.describe Cart do

  let(:cart)     { Cart.create }
  let(:book_one) { products(:ruby) }
  let(:book_two) { products(:two) }

  before do
    cart.add_product(book_one).save!
    cart.add_product(book_two).save!
  end

  it "can have multiple products added" do
    expect(cart.line_items.size).to eq(2)
  end

  it "calculates the total price of all products" do
    expect(cart.total_price).to eq(book_one.price + book_two.price)
  end
end
```

It almost looks like a different programming language! Developers who prefer RSpec like that the test reads like English: "Describe Cart, it can have multiple products added, expect `cart.line_items.size` to eq 2."

We're going to quickly go through how to write tests in RSpec without too much explanation. A great book for that is already available—*Effective Testing with RSpec 3* [MD17]—so we'll learn just enough RSpec to see it working with Rails, which demonstrates Rails' configurability. Although many developers who use RSpec set it up from the start of a project, you don't have to. RSpec can be added at any time, and that's what we'll do here.

Add rspec-rails to your `Gemfile`, putting it in the development and test groups: